

# DraftFM: Implementation and Reproducibility Reference

Brian Ward

## Abstract

This is the companion reference for “DraftFM: Zero-Shot Drafting of Unseen *Magic: The Gathering* Sets from Public Card Features.” The main paper is the scientific story: modeling decisions, the pre-registered protocol, results, and an honest self-assessment of what it does and doesn’t show. This document is the short, structured “how it’s actually built and reproduced” reference instead – released code layout, the provenance mechanics behind every number in the main paper, and the frozen identifiers that pin the evaluation. Read the main paper first.

## 1 What’s released

Training and evaluation code, the frozen protocol document and metric implementations, data and featurizer manifests, model weights for every battery member, and the per-pick prediction files from which every statistic in the main paper is computed (archived on Zenodo). Raw 17Lands bytes for the MSH test snapshot are deposited only after a courtesy note to 17Lands; every other input is re-derivable from public sources via the manifests below.

## 2 Provenance mechanics

Every training and evaluation run appends one record to an append-only ledger: run id, git sha, protocol tag, seed, framework version, full config, data-manifest hash, wall clock, throughput, artifact sha256s, and dev metrics. Every table cell in the main paper carries its source run id as a comment in the generated table file; tables and the scaling figure are emitted by a script from run records and cached evaluation outputs. Primary table and headline numbers read from generated macros; secondary analysis numbers (per-set deltas, diagnostic slices) are cited in prose with the exact producing script and run id in a source comment – no number anywhere is typed by hand without that trail, macro or not.

Training’s MPS backend is not trusted by default: every launch runs a CPU-vs-accelerator parity check on losses and gradient norms before training proceeds, LayerNorm is composed from primitive ops rather than the fused kernel (whose backward pass is wrong on some chip/OS combinations), and a watchdog skips non-finite gradient steps during training and halts on recurrence rather than silently continuing.

- `experiments/ledger.jsonl` – the append-only run registry
- `scripts/make_paper_tables.py` – emits every table and the scaling-figure data from ledger + run records; regenerate after any new run lands

- `figures/make_verdict_panels.py` – computes the real, live-scored numbers in the illustrative verdict-panel figures via the same `mtga.models.registry / mtga.draft_api.HUB` code path the deployed overlay uses
- `figures/fetch_example_cards.py` – fetches the card images used in those figures directly from Scryfall’s hosted URLs, writing `figures/cards/manifest.json` with the exact source URL, artist, and copyright line per card

### 3 Frozen identifiers

---

|                                  |                                     |
|----------------------------------|-------------------------------------|
| Protocol tag                     | <code>eval-protocol-v1.1</code>     |
| Data-manifest content hash       | <code>443bd25f72ee</code>           |
| Featurizer-manifest content hash | <code>e1313cadd03b</code>           |
| F-dev run id                     | <code>20260704_135822_f_dev</code>  |
| F-full run id                    | <code>20260705_110743_f_full</code> |

---

The MSH snapshot sha256 and ETag are recorded at freeze time and reported here: [PENDING: MSH snapshot sha256, ETag, download timestamp, gate report].